To Think or Not to Think: Evaluating LLM Reasoning and Agents in Vulnerability Detection

Hua Beng Tan
Singapore Management University
Singapore, Singapore
huabeng.tan.2024@engd.smu.edu.sg

Ratnadira Widyasari
Singapore Management University
Singapore, Singapore
ratnadiraw.2020@phdcs.smu.edu.sg

Merve Astekin
SINTEF
Oslo, Norway
merve.astekin@sintef.no

Arda Goknil
SINTEF
Oslo, Norway
arda.goknil@sintef.no

Hasan Sözer
Ozyegin University
Istanbul, Turkiye
hasan.sozer@ozyegin.edu.tr

Yan Naing Tun
Singapore Management University
Singapore, Singapore
yannaingtun@smu.edu.sg

Erik Johannes Husom
SINTEF
Oslo, Norway
erik.johannes.husom@sintef.no

Lwin Khin Shar
Singapore Management University
Singapore, Singapore
lkshar@smu.edu.sg

Lingxiao Jiang
Singapore Management University
Singapore, Singapore
lxjiang@smu.edu.sg

Abstract

Large language models (LLMs) are increasingly studied for software vulnerability detection, but the benefits of recent reasoning-enabled ("thinking") modes remain unclear. At the same time, agentic LLM designs aim to enhance analysis through structured collaboration, while practical software security workflows require explanations that facilitate inspection, validation, and remediation. Because prior studies often examine these dimensions separately or under heterogeneous settings, their individual effects on vulnerability detection remain difficult to disentangle. This paper addresses the gap through a phased, factor-separated empirical study on the PrimeVul Pair benchmark. In the first phase, we compare instruct and thinking variants of four open models under zero- and few-shot prompting without agents. In the second phase, we evaluate single-, dual-, and multi-agent (SA, DA, and MA) designs alongside call-matched single-agent baselines, alert-burden metrics, and cost analysis. In the third, we assess explanations through blinded human and calibrated LLM evaluation across correct and incorrect predictions, controlling for response length and model family. Thinking improves non-agent (NA) F1 in 7 of 8 settings, mainly through recall, while DA achieves the highest F1/recall but flags 88.2% of functions as vulnerable. MA performs better on pairwise discrimination, although call-matched self-revision achieves the best Pair-wise Correct Prediction (P-C) overall. Thinking also uses 2.7–4.4× more energy, and its F1 benefit declines with greater orchestration. Explanation gains are model-specific, partly length-mediated, and do not distinguish correct from incorrect predictions.

CCS Concepts

• **Software and its engineering**;



This work is licensed under a Creative Commons Attribution 4.0 International License.
ASE '26, Munich, Germany
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2882-2/2026/10
https://doi.org/10.1145/3832783.3837521

Keywords

LLM Reasoning, Vulnerability Detection, Agents, Thinking Mode

ACM Reference Format:
Hua Beng Tan, Ratnadira Widyasari, Merve Astekin, Arda Goknil, Hasan Sözer, Yan Naing Tun, Erik Johannes Husom, Lwin Khin Shar, and Lingxiao Jiang. 2026. To Think or Not to Think: Evaluating LLM Reasoning and Agents in Vulnerability Detection. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26), October 12–16, 2026, Munich, Germany*. ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3832783.3837521

1 Introduction

Large language models (LLMs) have become central to automated software engineering, supporting tasks such as code understanding, synthesis, repair, and security analysis [4, 18, 24, 28–30, 37, 50]. In software security, this trend has been increasingly applied to vulnerability detection, where recent studies [3, 6, 8, 14, 16, 20, 22, 25, 33, 39–41, 54, 55, 62, 68, 69] show that LLMs can be competitive under prompt-based settings, yet reveal substantial limitations under more realistic and demanding evaluation conditions. More broadly, evidence from code analysis suggests that LLMs can support security-oriented reasoning, although their effectiveness remains highly sensitive to task formulation, dataset quality, and evaluation protocol [19].

This context makes vulnerability detection an especially important testbed for current LLM design choices. Unlike many generic code tasks, vulnerability detection requires models to distinguish subtle semantic differences between secure and insecure implementations, reason about security-relevant behavior, and communicate decisions in a form that developers can inspect and act on [19, 42]. Recent benchmark work has sharpened this challenge [2, 15, 44, 48, 62]. Ding et al. [15] show that earlier vulnerability datasets can overestimate model capability because of duplication, label-quality issues, and unrealistic evaluation conditions, and introduce PrimeVul to better reflect real-world vulnerability detection settings. More recent benchmarking further indicates that LLM performance in practical vulnerability discovery remains far from solved [62].

Two recent developments further motivate a closer empirical study. First, modern LLMs increasingly provide explicit **reasoning-enabled** or **thinking** modes, extending prior work showing that intermediate reasoning can improve performance on complex tasks; for example, chain-of-thought prompting demonstrated that stepwise reasoning can substantially improve results on difficult reasoning benchmarks [53]. Second, **agentic** LLM systems have become a prominent design pattern in software engineering, allowing models to plan, critique, revise, and interact with tools or other agents rather than producing a single one-shot output [28, 32]. Prior work on SWE-agent and related software-engineering agents suggests that such orchestration can significantly affect task outcomes, while also introducing additional complexity and overhead [58, 60].

At the same time, software security practice requires more than accurate labels [17, 21]. A vulnerability prediction is far more useful when accompanied by an explanation that helps developers understand the risk, assess the supporting evidence, and guide inspection or remediation. Yet explanation quality is inherently multidimensional [10, 23, 36]. Prior interpretability research shows that fluent or plausible explanations are not necessarily faithful to the actual basis of a model's decision, and that explanation evaluation must distinguish readability and usefulness from grounding and evidential support [13, 31]. This distinction is particularly important in vulnerability detection, where persuasive but weakly grounded explanations may foster misplaced trust.

These developments reveal a central problem in the current literature: the effects of **reasoning capability**, **agentic orchestration**, and **explanation quality** are often entangled. Apparent improvements may stem from reasoning mode, reviewer-style collaboration, richer prompting, or simply longer and more elaborate outputs. Likewise, an explanation that seems better may reflect fluency rather than security-relevant grounding. This makes it difficult to identify which design choices genuinely improve vulnerability detection and which primarily add computational cost or stylistic variation. In security-sensitive settings, such ambiguity is especially problematic, as overstated model capability can lead to misplaced confidence in automated analysis.

To address this problem, this paper presents a **phased, factor-separated evaluation** of LLM-based vulnerability detection. We focus exclusively on vulnerability detection to obtain a tighter and more internally valid study than a broader multi-task design would allow. Using the **PrimeVul Pair benchmark** [15], we analyze how reasoning-enabled inference, agentic coordination, and explanation behavior interact under controlled conditions. Our study design is organized around three research questions (RQs):

- ***RQ1.*** Do reasoning-enabled LLMs outperform non-reasoning LLMs in software vulnerability detection?
- ***RQ2.*** How do multi-agent configurations compare to single-agent setups in terms of performance, cross-setting stability, and improvement consistency for vulnerability detection?
- ***RQ3.*** How does explanation quality differ between thinking and non-thinking LLMs in vulnerability detection?

Methodologically, the evaluation proceeds in three stages. First, we compare instruct- and thinking-mode variants of four open models (**Nemotron-Nano-8B**, **Nemotron-Super-49B**, **Qwen3-4B**, and **Qwen3-30B-A3B**) under non-agent (NA) execution with zero- and few-shot prompting. Second, we examine orchestration through single-agent (SA), dual-agent (DA) proposer–reviewer, and multi-agent (MA) review-pipeline designs, complemented by call-matched self-revision and best-of-(N) baselines, an alternative role workflow, and alert-burden and cost analyses. Third, we assess explanations under a blinded human and calibrated LLM-judge protocol spanning correct and incorrect predictions, while controlling for response length and testing model-family dependence.

Our results show that thinking mode improves NA F1 in 7 of 8 settings, with a statistically significant aggregate effect driven mainly by recall, while improving Pair-wise Correct Prediction (P-C) in 6 of 8 settings. Under orchestration, DA achieves the highest and most stable F1/recall but does so by flagging 88.2% of functions as vulnerable, whereas MA more often provides stronger pairwise discrimination at a lower false-positive rate. Call-matched experiments further show that four-call self-revision can equal or exceed MA P-C, indicating that part of the apparent collaboration benefit comes from additional inference budget. Thinking costs 2.7–4.4× more energy, and its F1 advantage weakens as orchestration increases. For explanations, humans corroborate a thinking-mode informativeness benefit for Nemotron, but the automatic judge's broader advantage is substantially length-mediated and does not generalize to Qwen3-30B. Blinded evaluations show that fluent explanations do not signal whether their predictions are correct.

This paper makes the following four contributions: a controlled statistical and pairwise evaluation of thinking modes across four open models; an analysis of agentic orchestration covering performance, stability, alert burden, and reasoning interactions; call-matched baselines and cost measurements that separate collaboration from inference budget; and a blinded, length-controlled, model-aware explanation study showing that explanation quality does not imply prediction reliability.

2 Related Work

LLMs for Software Vulnerability Detection. Recent studies identify LLMs as promising for vulnerability detection but show that measured effectiveness depends strongly on benchmark quality and evaluation protocol. Early prompt-based work [69] suggested that general-purpose LLMs can capture vulnerability-relevant patterns, whereas later benchmarks revealed inflated performance caused by duplication, noisy labels, and unrealistic settings. Ding et al. [15] demonstrate the impact of benchmark design, while Yildiz et al. [62] report substantial degradation in interprocedural, repository-level settings. Broader evidence likewise shows uneven robustness and generalization [19]. Prior work establishes LLM viability, but rarely examines how reasoning mode, agentic configuration, and explanation quality jointly affect vulnerability detection.

Reasoning-Enabled LLMs for Code and Security Analysis. Reasoning-enabled LLMs are motivated by evidence that explicit intermediate reasoning can improve complex-task performance. Chain-of-thought prompting [53] suggests similar benefits for vulnerability detection, which depends on security-relevant semantics rather than shallow lexical cues. However, such gains are not guaranteed: performance remains uneven under demanding generalization settings [19], benchmark quality and evaluation protocol strongly affect conclusions [15], and results decline in

repository-level settings requiring richer context [62]. Thus, reasoning is promising for code and security analysis, but it remains unclear whether thinking-enabled models outperform non-thinking variants under controlled vulnerability-detection conditions.

Agentic LLMs for Software Engineering and Security. Recent work suggests that agentic orchestration can improve LLM performance by replacing one-shot generation with structured reasoning, action, review, and revision [28, 32, 34, 51]. ReAct [61] established this pattern for multi-step problem solving, while systems such as RepairAgent [7] applied it to software engineering through planning, evidence gathering, tool use, and iterative refinement. In software security, Yildiz et al. [62] show that ReAct-style agents with contextual retrieval can outperform plain prompting, though performance remains inconsistent in repository-level settings. Thus, agentic vulnerability analysis is promising but insufficiently understood, and prior work lacks a controlled comparison of non-agent, single-agent, dual-agent, and multi-agent configurations under matched models, reasoning modes, and prompts.

Explanation Quality in LLM-Based Vulnerability Analysis. As LLMs enter security-sensitive software tasks, explanation quality matters alongside accuracy [65]. Prior work shows that explanation quality is multidimensional [10, 23, 36]: fluent explanations may be plausible yet unfaithful to the model's actual reasoning [31], while evidence-based frameworks such as ERASER emphasize grounding in supporting artifacts [13]. Recent studies distinguish dimensions such as faithfulness, completeness, and overall quality [10]. In software engineering, explanations are useful only when they help developers inspect, validate, and act on predictions [46], and explainable-AI research similarly stresses actionability and practical value over readability alone [5, 9, 11, 38, 43, 59, 63]. Yet controlled evidence comparing explanation quality between thinking and non-thinking LLMs in vulnerability detection remains limited.

3 Study Design

This section presents the study design, including the dataset, models, variables, research questions, and methodology (see Table 1).

3.1 Research Questions

RQ1 examines whether **thinking-mode** models outperform their **instruct-mode** counterparts in a **non-agent setup** under **zero-shot** and **few-shot prompting**. It isolates the contribution of explicit reasoning without agentic collaboration, using standard vulnerability-detection metrics.

RQ2 investigates whether **agentic orchestration** improves vulnerability detection beyond simpler execution settings. To answer this question, we compare **single-agent (SA)**, **dual-agent (DA)**, and **multi-agent (MA)** configurations under the same model, reasoning-mode, and prompting settings.

RQ3 analyzes how the quality of explanations differs between thinking and non-thinking models. Explanations are evaluated along four dimensions (**completeness**, **informativeness**, **clarity**, and **actionability**) using both manual and automatic assessment.

3.2 Vulnerability Detection Task and Dataset

We study **software vulnerability detection** as a binary classification task in which an LLM receives a function and predicts whether it is **vulnerable** or **benign**. Specifically, we utilize a high-quality and challenging dataset for vulnerability detection, PrimeVul Pair [15], comprising 870 balanced instances (435 vulnerable and 435 benign functions). PrimeVul Pair is particularly well suited to our study because each vulnerable function is paired with a benign counterpart sharing at least 80% of its string content. This construction reduces the effectiveness of superficial lexical matching and makes successful classification more dependent on finer-grained semantic reasoning about security-relevant code behavior. Although the benchmark is constructed from vulnerable–benign pairs, each function is treated as an individual input instance during inference, and the paired counterpart is not provided to the model. This allows us to evaluate vulnerability detection as a balanced classification problem while preserving the benchmark's core challenge of distinguishing highly similar secure and insecure implementations.

Table 1: Experimental design for vulnerability detection.

Factor	Levels / Description
Models	Nemotron-Nano-8B, Nemotron-Super-49B, Qwen3-4B, Qwen3-30B-A3B
Modes	Instruct-mode (non-reasoning), Thinking-mode (reasoning-enabled)
Prompting strategies	Zero-shot, Few-shot
Agent configurations	Single-agent (SA); Dual-agent (DA, proposer–reviewer); Multi-agent (MA, four-agent pipeline with moderator and review board)
RQ1 design	4 models × 2 modes × 2 prompting strategies × non-agentic configuration (direct LLM query, no state) = 16 experiments
RQ2 design	4 models × 2 modes × 2 prompting strategies × 3 agent configurations (SA, DA, MA) = 48 experiments
RQ3 design	Explanations analyzed for the same configurations used in RQ1, comparing instruct-mode and thinking-mode outputs in terms of completeness, informativeness, clarity, and actionability
Total configurations	16 (RQ1) + 48 (RQ2) = 64 experimental configurations, with RQ3 conducted on the generated outputs

PrimeVul Pair is appropriate for all three research questions addressed in this paper. For **RQ1**, it enables a controlled comparison between instruct-mode and thinking-mode models on a task that requires semantic discrimination rather than shallow pattern recognition. For **RQ2**, it provides a meaningful basis for evaluating whether proposer–reviewer and multi-agent pipelines improve detection accuracy and robustness by supporting structured critique and refinement. For **RQ3**, the function-level setting facilitates analysis of explanation quality, since model justifications can be examined in relation to specific code fragments and vulnerability-related reasoning. A limitation of the dataset is that it focuses on function-level vulnerabilities and therefore does not capture broader project-level or inter-procedural security contexts; however, it offers a suitable and challenging benchmark for controlled empirical comparison.

3.3 Models

The study employs four open LLMs: **Qwen3-4B**, **Qwen3-30B-A3B**, **Nemotron-Nano-8B**, and **Nemotron-Super-49B**. These models were selected to provide diversity in both model family and model scale, while also supporting a controlled comparison between instruct-mode and thinking-mode operation. This makes them well suited for the objectives of the study, which require examining the effect of reasoning capability within the same model family rather than across unrelated models only.

A key requirement of the experimental design is the ability to compare **non-reasoning** and **reasoning-enabled** behavior under otherwise comparable conditions. For this reason, each model is evaluated in two variants: **instruct-mode**, which serves as the

non-reasoning baseline, and **thinking-mode**, which serves as the reasoning-enabled variant. The instruct-mode configuration is intended to produce direct task outputs with no explicit extended reasoning process, whereas the thinking-mode configuration allows the model to engage in more deliberate reasoning before generating its final answer. This paired setup forms the foundation for **RQ1**, which investigates whether reasoning-enabled LLMs outperform non-reasoning LLMs in software engineering tasks.

The selected models also cover a range of capacities, from relatively compact models to substantially larger ones. Including both the **Nemotron** and **Qwen** model families reduces the likelihood that the findings are tied to a single architecture lineage or vendor ecosystem. At the same time, the variation in model size enables analysis of whether the effects of reasoning, prompting, and agentic orchestration remain stable across scales or become more pronounced in larger models.

3.4 Implementation and Hardware Details

All models were served using vLLM (v0.17–v0.18), an open-source inference engine that provides an OpenAI-compatible API. Each model was loaded with a standardized context window of 65,536 tokens (64K) and served on NVIDIA H100 80GB HBM3 GPUs via RunPod cloud infrastructure. To ensure reproducibility and eliminate stochastic variation in outputs, all experiments were conducted with the temperature parameter fixed at 0, following prior work [55]. In our own pilot repetitions under this setting, we observed no variation in the outputs consistent with the previous study [55]; therefore, we report a **single run per configuration**. Agent construction was performed using the AG2 framework [1], an open-source platform for building multi-agent systems.

3.5 System Configurations

To systematically evaluate the impact of agentic collaboration on vulnerability-detection accuracy, we implemented four progressively collaborative configurations. Designing multi-agent LLM systems involves a large design space, including the number of agents, their roles, and their communication and coordination strategies; accordingly, there is no single universally optimal multi-agent architecture for vulnerability detection. In this study, our objective is not to propose a new task-specific multi-agent design, but rather to evaluate how representative agentic configurations influence vulnerability-detection performance under controlled conditions. The implementations used in our experiments instantiate representative role configurations inspired by patterns previously reported in the literature for vulnerability detection [55], enabling a controlled comparison of how increasing collaboration affects performance, cross-setting stability, and improvement consistency in our study, rather than claiming a task-optimal architecture.

- *Non-Agentic (NA)*: This involves direct calls to the LLM without maintaining any state or exhibiting agent-like behavior.
- *Single Agent (SA)*: A single LLM-based agent acts autonomously to complete the task.
- *Dual Agent (DA)*: This includes two LLM-based agents with task-specific roles. A Code Author proposes an analysis of the function, and a Security Analyst reviews the Code Author's reasoning and the code itself to make the final vulnerability decision.
- *Multi-Agent (MA)*: Four agents are involved in a coordinated setup with task-specific roles [55]. A Security Researcher analyzes the code for vulnerabilities, a Code Author responds to the Security Researcher's findings, a Moderator summarizes the discussion, and a Review Board makes the final vulnerability decision.

These configurations represent distinct levels of agentic interaction and orchestration [35, 47]. The **NA** setup captures the simplest use of an LLM as a direct classifier, without any explicit orchestration, aligning with the *one-shot model querying* [35] pattern. It serves as a minimal non-agentic baseline and therefore provides the cleanest reference point for assessing the effect of reasoning mode alone. The **SA** setup wraps the same LLM in an agent abstraction that manages prompts, execution flow, and internal state; although **NA** and **SA** both rely on a single LLM and are therefore expected to show similar task performance, distinguishing them allows us to isolate the effect of adopting an agentic framework from the additional effect of adding inter-agent collaboration. The **DA** configuration introduces competitive collaboration through debate-like review and cross-reflection [35, 47], aiming to improve reasoning consistency and output reliability at the cost of additional communication overhead and energy consumption. The **MA** employs role-based and voting-based cooperation [35, 47], representing a more collaborative division of labor that may yield more refined and robust outcomes, albeit with greater coordination overhead.

3.6 Prompting Strategies

We use two prompting strategies throughout the study: zero-shot and few-shot. In both cases, vulnerability detection is framed as a **binary classification task** over a single input function, requiring the model to predict whether the function is vulnerable or benign. All prompts follow the same basic structure: a task instruction, the input code snippet, an output-format constraint requiring a binary label, and, when applicable, a brief justification. In the **few-shot setting**, the prompt additionally includes a small set of representative labeled examples before the target function. The same demonstration examples are used across all models and reasoning modes.

All prompt templates can be found in the replication package [45]. Across all settings, we keep the prompt wording, label space, and response-format constraints as consistent as possible, varying only the information needed by the orchestration pattern.

3.7 Variables

3.7.1 *Independent Variables (Factors We Manipulate).*

(1) **Model** (Nemotron-Nano-8B, Nemotron-Super-49B, Qwen3-4B, and Qwen3-30B-A3B) captures differences across model families and scales.
(2) **Reasoning mode** (instruct-mode and thinking-mode) represents the central comparison in the study, contrasting non-reasoning behavior in instruct-mode with reasoning-enabled behavior in thinking-mode.
(3) **Prompting strategy** (zero-shot and few-shot) allows us to examine whether the observed effects remain stable when the models are provided with no demonstrations or with a small number of examples.

(4) **System configuration** (NA, SA, DA, and MA) captures the system-level orchestration design, ranging from a single model instance to multi-agent review pipelines.

3.7.2 Dependent Variables (Measured Outcomes).

(1) **Vulnerability detection performance** captures how accurately a configuration classifies a code function as vulnerable.
(2) **Cross-setting stability (consistency)** captures how stable and consistently beneficial a configuration is across experimental settings. We assess cross-setting stability and improvement consistency through the stability of performance across models, prompting strategies, and reasoning modes, and through the consistency of improvement relative to simpler baselines.
(3) **Explanation quality** captures the quality of the explanations generated by the models in different configurations and is analyzed along four dimensions: completeness, informativeness, clarity, and actionability.

3.8 Evaluation Metrics

For **RQ1** and **RQ2**, we treat vulnerability detection as binary classification and report accuracy, precision, recall, and F1, capturing overall correctness, prediction reliability, vulnerability coverage, and their balance. Because the dataset is balanced, these metrics are directly interpretable. Following Ding et al. [15], we report Pairwise Correct Prediction (P-C), which counts a vulnerable–benign pair as correct only when both functions are classified correctly, measuring discrimination between highly similar code instances.

For **RQ2**, we assess each agent configuration's cross-setting stability and improvement consistency. Stability is measured by mean F1, standard deviation, and coefficient of variation across models, prompts, and reasoning modes, with lower variability indicating greater stability. Improvement consistency is the proportion of matched settings in which a configuration outperforms the single-agent baseline in F1.

For **RQ3**, following prior work [55], we evaluate explanations on **completeness**, **informativeness**, **clarity**, and **actionability**. These capture whether an explanation covers the prediction rationale, provides specific security-relevant insight, is coherent and understandable, and supports inspection, validation, or remediation. Each dimension is rated on a five-point Likert scale [12, 49, 55, 56, 67].

3.9 Methodology

To ensure fairness and reproducibility, all configurations were evaluated on the same PrimeVul Pair dataset using the same task formulation, prompt structure, and output-parsing protocol, except where explicitly varied by the factor under study. In every experiment, the input consisted of a single function, and the model was asked to classify it as vulnerable or benign and, when requested, justify its decision. The paired counterpart in PrimeVul Pair was not shown during inference. Across models and settings, we used the same label space, response format, and zero-shot and few-shot prompt templates. In the few-shot setting, the same demonstration examples were used for all models and modes. Predictions were automatically parsed into binary labels, and non-conforming outputs were handled conservatively in the analysis. The source code, prompts, and dataset are provided in the replication package [45].

3.9.1 RQ1. To answer RQ1, we compared instruct- and thinking-mode variants of Nemotron-Nano-8B, Nemotron-Super-49B, Qwen3-4B, and Qwen3-30B-A3B under the NA setup. Each model–mode combination was evaluated with both zero-shot and few-shot prompting, yielding 16 configurations for vulnerability detection. The aim was to isolate the effect of explicit reasoning at the model level while keeping orchestration fixed.

3.9.2 RQ2. To answer RQ2, we evaluated SA, DA, and MA across four models, two reasoning modes, and two prompting strategies, yielding 48 configurations. All designs used the same dataset, task instruction, output constraints, and binary label space, so differences reflect orchestration. In DA, a reviewer validates the proposer's prediction and justification; in MA, specialized agents review the initial analysis and a moderator produces the final verdict. All final outputs were mapped to the same binary label space as the SA baseline, ensuring direct metric comparability.

3.9.3 RQ3. To answer RQ3, we compared Nemotron-Super-49B explanations in thinking and instruct modes, as it performed best in RQ1 (Section 4.1). Under a blinded protocol, two annotators rated 120 explanations from 60 snippets—15 each from TP, TN, FP, and FN—with every snippet evaluated in both modes. Ground truth, CWE, CVE description, model, and mode were withheld, and annotators received detailed guidelines and calibration examples. The paired tests in Section 4.3 use 59 matched thinking/instruct pairs for both human and automatic evaluations; one snippet was excluded because a duplicate PrimeVul instance lacked a paired judge score in one mode.

We complemented manual evaluation with Claude Sonnet 4.6 as an independent LLM judge, following prior LLM-as-a-judge and SE studies [26, 27, 54]. Chosen outside the Qwen3 and Nemotron families to reduce self-evaluation bias [55], it used the same blinded rubric as human raters, with ground truth, CWE, and CVE withheld. Because the compressed 1–5 scales are sensitive to calibration offsets, we used few-shot exemplars from held-out human ratings. Five-fold cross-validation assessed adjacent (±1) agreement, mean absolute error, and Spearman's ρ [64] against the inter-human ceiling, which the calibrated judge reached on all dimensions. Neither another judge (Claude Opus 4.8) nor consensus-based calibration improved alignment. Thus, automatic evaluation complements rather than replaces human judgment and supports triangulation [55, 66].

For analysis, we aggregated manual ratings by averaging annotator scores for each explanation and metric. We then compared explanation quality between thinking and instruct modes, for both manual and automatic evaluations, using the Wilcoxon signed-rank test [57], which is appropriate for paired ordinal data without assuming normality.

4 Results

4.1 *RQ1. Reasoning Effects in Non-Agent (Direct LLM) Vulnerability Detection*

Table 2 and Figure 1 summarize the NA results. Thinking mode outperforms instruct mode in 7 of 8 model–prompting combinations, with the exception being Qwen-30B-A3B zero-shot, where instruct is slightly better (F1 = 0.533 vs. 0.508). The gains nevertheless vary substantially across models. The aggregate advantage is statistically

Table 2: NA vulnerability detection results on PrimeVul Pair (870 samples) across four models, two modes, and two prompting strategies. Best F1 and P-C per model are bold.

Family	Model	Mode	Prompt	Accur.	Prec.	Recall	F1	P-C
Nemotron	Nano-8B	Instruct	Few	.497	.487	.090	.151	2.9
		Instruct	Zero	.499	.500	.005	.009	0.5
		Thinking	Few	.493	.487	.223	**.306**	**4.4**
		Thinking	Zero	.506	.583	.048	.089	1.5
	Super-49B	Instruct	Few	.528	.547	.336	.416	18.6
		Instruct	Zero	.543	.574	.338	.425	19.3
		Thinking	Few	.538	.532	.655	.587	**19.8**
		Thinking	Zero	.540	.529	.749	**.620**	19.3
Qwen	4B	Instruct	Few	.509	.541	.136	.217	9.3
		Instruct	Zero	.522	.576	.175	.268	9.0
		Thinking	Few	.510	.526	.234	.324	12.7
		Thinking	Zero	.529	.552	.315	**.401**	**18.6**
	30B-A3B	Instruct	Few	.497	.497	.423	.457	16.6
		Instruct	Zero	.550	.555	.513	**.533**	**22.5**
		Thinking	Few	.529	.538	.425	.475	17.8
		Thinking	Zero	.536	.542	.478	.508	19.6

significant and practically large: across the eight NA configurations, thinking improves mean F1 from 0.31 to 0.41 (Wilcoxon $p = 0.023$, Cohen's $d = 1.37$). The gain is driven mainly by recall, as accuracy differs only marginally (0.523 vs. 0.518, $p = 0.46$, $d = 0.35$), indicating a shift in the false-negative/false-positive balance rather than overall agreement. Therefore, we avoid over-interpreting individual cells, including Qwen3-30B-A3B zero-shot, where instruct slightly outperforms thinking (F1 0.533 vs. 0.508).

The strongest model is Nemotron-Super-49B. Its instruct-mode achieves F1 = 0.416 with few-shot prompting and 0.425 with zero-shot prompting, while thinking-mode increases these scores to 0.587 and 0.620, respectively. This makes **thinking zero-shot Nemotron-Super-49B** the best overall configuration in terms of F1, as also reflected by the heatmap in Figure 1. These gains are driven primarily by a large increase in recall, which rises from ~0.34 in instruct-mode to 0.655 and 0.749 in thinking-mode, indicating a much stronger ability to identify vulnerable functions. The improvement in accuracy, by contrast, is modest, suggesting that reasoning mainly changes the balance between false negatives and false positives rather than overall label agreement alone.

The Qwen models show a similar pattern. For Qwen3-4B, thinking raises F1 from 0.217 to 0.324 with few-shot prompting and from 0.268 to 0.401 with zero-shot prompting. For Qwen3-30B-A3B, the gain is smaller: F1 increases from 0.457 to 0.475 in few-shot mode, while zero-shot instruct slightly outperforms thinking (0.533 vs. 0.508). Thus, reasoning substantially benefits Qwen3-4B but not the larger Qwen3-30B-A3B under zero-shot prompting.

The weakest model is **Nemotron-Nano-8B**. Thinking-mode improves F1 from 0.151 to 0.306 in the few-shot setting and from 0.009 to 0.089 in the zero-shot setting, so the directional benefit remains positive. However, the absolute performance remains poor, especially in zero-shot mode, where recall is extremely low in both instruct and thinking settings. This indicates that reasoning alone is insufficient to compensate for limited model capacity.

The effect of **prompting strategy** is also more nuanced than a simple few-shot-versus-zero-shot conclusion. For **Nemotron-Super-49B**, **Qwen3-4B**, and **Qwen3-30B-A3B**, **zero-shot** yields higher F1 than few-shot in both instruct and thinking modes. In contrast, for **Nemotron-Nano-8B**, **few-shot** clearly outperforms zero-shot in both modes. The heatmap makes this pattern easy to see: most rows show stronger performance in the zero-shot column, except for the 8B model, where few-shot is superior. This suggests that the usefulness of demonstrations depends on model capacity and is not uniformly beneficial.

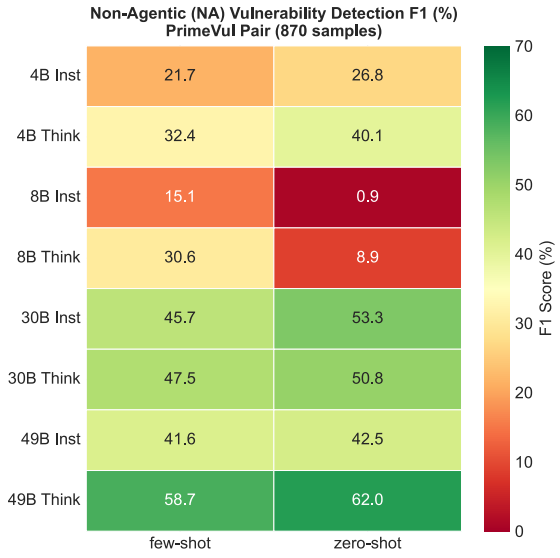


Figure 1: NA vulnerability detection F1 heatmap across models and configurations. Rows represent model–mode combinations; columns represent prompting strategies.

The **P-C metric** provides a complementary view: thinking improves P-C in 6 of 8 model–prompting combinations. Gains occur for Nemotron-Nano-8B (2.9⟶4.4 few-shot; 0.5⟶1.5 zero-shot), Qwen3-4B (9.3⟶12.7; 9.0⟶18.6), and Qwen3-30B-A3B few-shot (16.6⟶17.8). The exceptions are Nemotron-Super-49B zero-shot, which is essentially unchanged at 19.3, and Qwen3-30B-A3B zero-shot, where instruct is higher (22.5 vs. 19.6). The best overall P-C is Qwen3-30B-A3B instruct zero-shot (22.5), followed by Nemotron-Super-49B thinking few-shot and Qwen3-30B-A3B thinking zero-shot (both about 20). Thus, thinking improves pair-wise discrimination in most, but not all, settings.

RQ1 Conclusion. Thinking mode generally improves non-agent vulnerability detection, outperforming instruct mode in 7 of 8 model–prompt combinations and increasing mean F1 from 0.31 to 0.41 ($p = 0.023$, $d = 1.37$). The gains are driven mainly by higher recall rather than accuracy. Nemotron-Super-49B thinking zero-shot achieves the best F1 (0.620), while Qwen3-30B-A3B instruct zero-shot achieves the best P-C (22.5). Thinking also improves P-C in 6 of 8 settings, although benefits vary substantially by model and prompting strategy.

4.2 *RQ2. Agentic Effects in Vulnerability Detection*

Table 3, Table 4, Figure 2, and Figure 3 compare the three agentic designs (SA, DA, and MA) across all model, reasoning-mode, and prompting combinations. The results show that **the agentic configuration changes the error profile of vulnerability detection substantially, but not uniformly**. We attribute these differences to the configuration as a whole rather than to collaboration *per se*:

Table 3: Vulnerability detection results across three agentic designs (SA, DA, MA) on PrimeVul Pair (870 samples). Each cell group shows Acc. / Prec. / Rec. / F1 / P-C for one design.

				SA					DA					MA				
Family	**Model**	**Mode**	**Prom.**	**Acc.**	**Pr.**	**Re.**	**F1**	**P-C**	**Acc.**	**Pr.**	**Re.**	**F1**	**P-C**	**Acc.**	**Pr.**	**Re.**	**F1**	**P-C**
Nem.	Nano-8B	Inst.	Few	.497	.487	.086	.146	2.7	.502	.501	.968	.660	3.2	.480	.485	.579	.528	17.4
		Inst.	Zero	.498	.333	.002	.005	0.2	.500	.500	.933	.651	3.2	.495	.468	.053	.095	4.2
		Think	Few	.503	.510	.241	.327	5.9	.506	.503	.961	.660	2.7	.525	.523	.597	.558	19.3
		Think	Zero	.502	.541	.046	.085	1.5	.502	.501	.943	.654	3.4	.499	.526	.079	.138	5.9
	Sup-49B	Inst.	Few	.516	.528	.326	.403	16.1	.503	.502	.959	.659	3.7	.503	.502	.851	.631	9.8
		Inst.	Zero	.541	.566	.363	.443	21.0	.501	.501	.984	.664	2.2	.480	.484	.600	.536	19.1
		Think	Few	.529	.524	.651	.581	18.9	.518	.510	.917	.656	9.5	.529	.522	.678	.590	21.0
		Think	Zero	.536	.524	.744	.615	19.2	.500	.500	.952	.656	3.9	.494	.493	.425	.457	20.5
Qwen	4B	Inst.	Few	.522	.581	.167	.259	11.2	.503	.502	.982	.664	1.5	.498	.499	.816	.619	10.0
		Inst.	Zero	.527	.587	.188	.284	11.5	.506	.503	.982	.665	2.4	.471	.480	.680	.562	15.2
		Think	Few	.534	.579	.255	.354	16.9	.503	.504	.421	.459	17.4	.499	.497	.415	.453	20.1
		Think	Zero	.524	.546	.301	.388	15.2	.503	.502	.818	.622	12.0	.520	.530	.334	.410	20.8
	30B	Inst.	Few	.532	.539	.464	.499	20.3	.495	.498	.984	.661	0.7	.526	.513	.930	.662	9.8
		Inst.	Zero	.532	.538	.471	.502	20.8	.498	.499	.995	.665	0.7	.528	.520	.715	.602	20.1
		Think	Few	.531	.541	.421	.473	18.1	.509	.507	.634	.564	18.8	.522	.516	.649	.575	19.1
		Think	Zero	.538	.543	.497	.519	20.5	.490	.493	.729	.588	14.7	.516	.514	.548	.531	21.5
Mean				**.523**	**.529**	.326	.368	13.7	.503	.502	**.885**	**.634**	6.2	.505	.505	.559	.497	**15.8**
Std				.014	.057	.203	.170	7.1	.006	.004	.161	.055	6.1	.018	.018	.241	.158	5.7
CV (%)							46	52				9	98				32	36

Table 4: Pairwise Correct Prediction (P-C, %) across three agentic designs on PrimeVul Pair.

Family	**Model**	**Mode**	**Prompt**	**SA**	**DA**	**MA**
Nemotron	Nano-8B	Inst.	Few	2.7	3.2	**17.4**
		Inst.	Zero	0.2	3.2	**4.2**
		Think	Few	5.9	2.7	**19.3**
		Think	Zero	1.5	3.4	**5.9**
	Super-49B	Inst.	Few	**16.1**	3.7	9.8
		Inst.	Zero	**21.0**	2.2	19.1
		Think	Few	18.9	9.5	**21.0**
		Think	Zero	19.2	3.9	**20.5**
Qwen	30B-A3B	Inst.	Few	**20.3**	0.7	9.8
		Inst.	Zero	**20.8**	0.7	20.1
		Think	Few	18.1	18.8	**19.1**
		Think	Zero	20.5	14.7	**21.5**
	4B	Inst.	Few	**11.2**	1.5	10.0
		Inst.	Zero	11.5	2.4	**15.2**
		Think	Few	16.9	17.4	**20.1**
		Think	Zero	15.2	12.0	**20.8**

SA, DA, and MA differ not only in agent structure but also in the number of inference calls, the amount of accumulated context, and the opportunities for self-revision. The call-matched single-agent baselines in Section 5.2 isolate the inference-budget component, and show that much of the apparent multi-agent benefit is recoverable by a single agent given the same call budget.

At the aggregate level, DA achieves the highest mean F1-score (0.634) and by far the highest mean recall (0.885), clearly exceeding both SA (F1 = 0.368, recall = 0.326) and MA (F1 = 0.497, recall = 0.559) (Table 3, Figure 2). However, this gain is not accompanied by higher accuracy or precision. In fact, SA retains the highest mean accuracy (0.523) and precision (0.529), whereas DA averages only 0.503 accuracy and 0.502 precision. This indicates that its F1 gain comes mainly from a recall-oriented shift, often at the cost of more false positives. MA lies between the two extremes, with mean F1 (0.497) well above SA and below DA; its accuracy (0.505) is close to SA while its precision (0.505) is marginally lower.

To quantify alert burden, we report false-positive rate (FPR) and predicted-positive rate (PPR). Across 16 configurations, DA flags 88.2% of functions (PPR = 0.882, FPR = 0.880), compared with SA

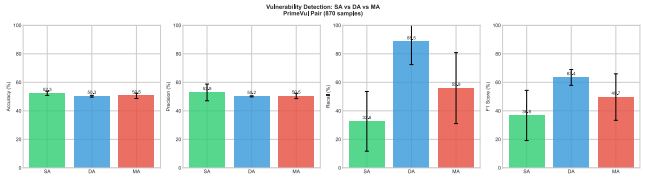


Figure 2: Vulnerability detection performance comparison across SA, DA, and MA designs on PrimeVul Pair (870 samples), grouped by model and reasoning mode.

at 30.3% (PPR = 0.303, FPR = 0.280) and MA at 55.4% (PPR = 0.554, FPR = 0.549). With precision near 0.50, DA produces roughly one false positive per true positive, so its higher F1 reflects a recall shift rather than better discrimination; it is therefore better suited to high-recall screening than stand-alone use. Nevertheless, the F1 gains over SA are statistically significant and practically large: over the 16 matched configurations (paired Wilcoxon signed-rank), DA vs. SA gives 0.634 vs. 0.368 ($p < 0.0001$, Cohen's $d = 1.40$) and MA vs. SA gives 0.497 vs. 0.368 ($p = 0.0013$, $d = 0.92$). Therefore, we treat F1/recall and P-C as complementary criteria rather than declaring one design universally best.

DA is the most consistently beneficial design when F1 is used as the primary metric. It outperforms SA in all 16 matched configurations. The strongest DA gains occur for weaker or mid-sized models. For Nemotron-Nano-8B, DA raises F1 from 0.005–0.327 under SA to a tightly clustered 0.651–0.660 across all four settings, effectively eliminating the severe failures observed in the non-collaborative setup. Similarly, for Qwen-4B, DA increases F1 from 0.259–0.388 under SA to 0.459–0.665, with especially large gains in the instruct-mode settings. For Qwen-30B-A3B, DA again outperforms SA in all four settings, improving F1 from 0.473–0.519 to 0.564–0.665. MA also outperforms SA in 15 of 16 settings and is more stable (F1 coefficient of variation - CV: 32% vs. 46%). Its mean F1 remains below DA's because it avoids DA's flag-everything operating point, while its main advantage appears in pairwise correctness.

The P-C results in Table 4 provide a complementary perspective. Unlike F1, which rewards recall improvements, P-C requires both



Figure 3: Vulnerability detection F1 heatmap across all 48 SA/DA/MA configurations on PrimeVul Pair (870 samples).

Table 5: Reasoning benefit (mean thinking − instruct) by orchestration level, on PrimeVul Pair (870 samples). Subscripts I/T denote instruct/thinking.

Design	$F1_I$	$F1_T$	ΔF1	P-C_I	P-C_T	ΔP-C
NA	.310	.414	+.104	12.3	14.2	+1.9
SA	.318	.418	+.100	13.0	14.5	+1.5
DA	.661	.607	−.054	2.2	10.3	+8.1
MA	.529	.464	−.066	13.2	18.5	+5.4

the vulnerable function and its paired benign counterpart to be classified correctly, making it more sensitive to fine-grained discrimination between highly similar code pairs. Under this metric, the picture changes markedly. MA yields the best P-C in 11 of the 16 settings, whereas SA is best in 5 and DA in none (mean P-C: MA 15.8, SA 13.7, DA 6.2). This is particularly evident in the few-shot and thinking settings — for Nemotron-Nano-8B, Qwen-4B, and Nemotron-Super-49B, MA is strongest in most few-shot and thinking configurations (MA P-C up to 21.5) — whereas SA remains competitive in several instruct zero-shot settings. By contrast, DA attains high recall but the poorest pair-wise correctness of the three designs — it is never the best design on P-C — indicating that its aggressive vulnerability predictions do not preserve the vulnerable–benign distinction required by P-C. Thus, while DA is the strongest design for maximizing F1, MA more often yields the best pair-wise discrimination.

The cross-setting stability statistics in Table 3 further differentiate the designs. DA has the lowest standard deviation and coefficient of variation for F1 (Std = 0.055, CV = 9%), compared with MA (Std = 0.158, CV = 32%) and SA (Std = 0.170, CV = 46%). Similar trends hold for accuracy and precision. This indicates that DA is not only stronger in average F1, but also the most stable configuration across models, prompting strategies, and reasoning modes. MA is more stable than SA (CV 32% vs. 46%), though less so than DA, and achieves the best P-C in most settings.

The benefit of thinking mode depends strongly on orchestration (Table 5). Thinking raises F1 substantially in NA and SA (+0.104 and +0.100), but the gain reverses under both DA (−0.054) and MA (−0.066), suggesting that individual-model reasoning is diluted across agents. For P-C, however, thinking remains beneficial in DA and MA (+8.1 and +5.4), indicating improved vulnerable–benign discrimination even without higher F1. This reasoning×orchestration interaction is promising but requires deeper per-model analysis and significance testing.

RQ2 Conclusion. Agentic orchestration helps unevenly. DA gives the highest, most stable F1/recall (beating SA in all 16 settings) but mainly by over-flagging—it labels ∼88% of functions vulnerable (FPR = 0.880) at lower precision, acting as a high-recall screen. MA is more stable than SA and best on pairwise discrimination (P-C in 11 of 16 settings) at a moderate FPR (0.549), making it preferable when vulnerable–benign discrimination matters. Thinking mode also interacts with orchestration: it helps F1 in NA and SA settings but reverses under both DA and MA, while still improving P-C in collaborative settings.

4.3 *RQ3. Explanation Quality across Thinking and Non-Thinking Models*

Table 6 reports explanation quality for Nemotron-Super-49B under a *blinded* protocol, in which the ground-truth label, CWE, and CVE description are withheld from both the human raters and the LLM judge (Sonnet-4.6, calibrated to the human ratings; see below). Blinding is necessary so that each explanation is rated on its own merits rather than on knowledge of whether the prediction was correct. Under this protocol, both evaluators agree that thinking-mode explanations are more *informative*, but the judge sees a broader advantage: it also rates thinking higher on completeness and clarity, which the blinded human raters do not corroborate (their completeness, clarity, and actionability means differ by at most 0.15 between modes, with no clear direction). The core informativeness advantage is thus human-corroborated, whereas the judge's additional completeness and clarity gains are specific to the automatic judge.

Paired Wilcoxon signed-rank tests (Nemotron-Super-49B, on 59 of the 60 thinking/instruct snippet pairs that make up the 120-explanation manual set; one snippet is excluded for a missing paired judge score, and both evaluators are tested on the same 59 pairs) refine this picture. The calibrated judge rates thinking-mode explanations significantly higher than instruct-mode on completeness (+0.49), clarity (+0.51), and informativeness (+0.64) (all $p < 0.0001$); actionability is higher but only marginally (+0.25, $p = 0.023$, not surviving Bonferroni correction). The human raters corroborate the *informativeness* advantage (+0.30, $p = 0.006$) but show no significant difference on the other three dimensions. Both evaluators, therefore, agree that thinking-mode explanations are more informative; the judge's additional completeness and clarity advantages are not corroborated by humans and, as the length analysis below shows, are substantially length-mediated. The judge thus reproduces the human-visible informativeness gain while over-attributing a broader thinking advantage.

Table 6: Explanation quality for Nemotron-Super-49B under the blinded protocol: mean score (1–5), reported as Human consensus / calibrated Sonnet-4.6 judge (H / J), by reasoning mode and by confusion-matrix stratum.

	Completeness (H / J)	Informativeness (H / J)	Clarity (H / J)	Actionability (H / J)
By reasoning mode				
Thinking	3.5 / 3.4	3.4 / 3.2	3.6 / 4.1	2.6 / 2.5
Instruct	3.6 / 2.9	3.1 / 2.6	3.5 / 3.6	2.8 / 2.2
By confusion stratum				
TP	3.2 / 3.1	3.2 / 3.0	3.5 / 3.8	3.1 / 2.9
TN	3.8 / 3.2	3.3 / 2.9	3.8 / 3.9	2.2 / 1.7
FP	3.6 / 3.1	3.4 / 3.0	3.5 / 3.7	3.4 / 3.0
FN	3.8 / 3.4	3.2 / 2.8	3.4 / 4.0	2.2 / 1.7
Overall	3.6 / 3.2	3.3 / 2.9	3.6 / 3.8	2.7 / 2.3

Across dimensions, both human and LLM evaluators consistently assign the highest scores to **clarity**, followed by completeness, informativeness, and actionability. **Actionability** receives the lowest scores in both evaluation settings, suggesting that, regardless of mode, the generated explanations are often difficult to translate into concrete follow-up actions.

We validate the calibrated judge (Sonnet-4.6, 24 few-shot exemplars drawn from held-out human ratings, blind-preserving) before extending it to the full 480-explanation frame. On this compressed 1–5 scale, the two human raters themselves agree only moderately by rank (ρ = 0.26–0.68, highest on actionability) yet within one point 96–98% of the time, so we benchmark agreement against this inter-human ceiling using MAE and adjacent (±1) agreement alongside ρ. Under 5-fold cross-validation, the calibrated judge agrees with the human ratings *as well as the two humans agree with each other* on all four dimensions (completeness ρ = 0.30 vs. ceiling 0.26; clarity 0.19/0.32; actionability 0.65/0.68; informativeness 0.30/0.28; MAE 0.51–0.64, adjacent 92–95%), and both order the dimensions identically.[1] Because it agrees at the human ceiling and reproduces the one mode-level effect humans corroborate (the informativeness advantage), we use the judge to score the full 480-explanation frame as a stand-in for human ratings; agreement is nonetheless weakest on false positives, and the judge's non-corroborated completeness/clarity thinking-preference is length-mediated, so we read those mode gaps with that caveat.

Figure 4 breaks the blind LLM-judge scores down by model and shows that the thinking-mode advantage is *model-specific*. For Nemotron-Super-49B, thinking exceeds instruct on completeness, clarity, and informativeness (as above); for Qwen3-30B, the two modes are statistically comparable, with thinking if anything marginally lower (differences ≤ 0.3). Together with the length effects and human–judge divergence in Table 6, this cautions against broadly claiming that thinking-mode explanations are better.

Because thinking-mode responses are far longer than instruct-mode responses (Nemotron-Super-49B, blinded judge, all four strata, n = 120 per mode: mean 13,579 vs. 2,935 characters, a 4.6× difference; 3,129 vs. 666 BPE tokens; Table 7a), we test whether the blind judge's thinking advantage is confounded by length. Length correlates positively with completeness, clarity, and informativeness overall (ρ = 0.30–0.38, all $p < 0.001$) but not with actionability (ρ = 0.10, n.s.; Table 7b). A partial Spearman correlation of mode on score *controlling for response length* shrinks the thinking-vs-instruct association by roughly 40–60% but does not eliminate it: completeness (ρ: 0.356 → 0.134, p = 0.039), clarity (0.359 → 0.204, p = 0.002), and informativeness (0.419 → 0.198, p = 0.002) retain a significant length-independent residual, while actionability shows no mode effect either way (0.134 → 0.089, n.s.). The blind judge's thinking-mode advantage on completeness, clarity, and informativeness is therefore *substantially but not wholly* length-mediated: longer, more elaborate explanations are rewarded, yet a modest quality signal survives length control. This partial mediation, together with the model-specificity above and the absence of a corresponding human advantage (Table 6), cautions against reading the judge's scores as a purely qualitative endorsement of thinking-mode explanations.

Table 7: Length-sensitivity analysis for the RQ3 LLM judge (Nemotron-Super-49B; calibrated blinded Sonnet-4.6 judge; four strata; $n \approx 120$ per mode): (a) response length, (b) score–length Spearman correlations, and (c) raw versus length-partialled mode–score correlations.

(a) Response length

Mode	Mean chars.	Median chars.	Mean BPE tokens	Median BPE tokens
Thinking	13,579	12,122	3,129	2,908
Instruct	2,935	2,854	666	635

(b) Score–length Spearman ρ (characters)

Split	Completeness	Informativeness	Clarity	Actionability
Overall	0.35	0.38	0.30	0.10
Thinking	0.00	−0.08	−0.16	−0.33
Instruct	0.22	0.28	0.21	0.35

(c) Mode→score: raw vs. length-partialled ρ

Dimension	Raw ρ	Partial ρ	p
Completeness	0.356	+0.134	0.039
Informativeness	0.419	+0.198	0.002
Clarity	0.359	+0.204	0.002
Actionability	0.134	+0.089	0.17

Explanation quality on false positives and negatives. The analysis above is restricted to correctly classified snippets. Because explanation quality on *incorrect* predictions is central to developer trust, we extend the LLM-judge evaluation to all four confusion strata (TP, TN, FP, FN) on a 480-explanation frame—240 code snippets from Nemotron-Super-49B and Qwen3-30B (SA zero-shot), each explained in both thinking and instruct modes (2 models × 2 modes × 4 strata × 30)—with a nested 120-explanation subset (Nemotron-Super-49B only) re-rated by two blinded human annotators. Critically, the judge is run *blind* to the ground-truth label, matching the blinded human protocol. Under this setup, explanation quality is statistically indistinguishable between correct (TP/TN, $n \approx 238$) and incorrect (FP/FN, $n \approx 237$) predictions on every dimension: the calibrated blind judge gives completeness 3.58 vs. 3.61, clarity 4.10 vs. 4.13, actionability 2.44 vs. 2.51, and informativeness 3.37 vs. 3.34 (Mann–Whitney, all $p > 0.3$), and no human dimension survives multiple-comparison correction (smallest p = 0.014 on clarity, above the Bonferroni threshold 0.0125). A fluent explanation for a wrong prediction reads as good as one for a correct prediction when the evaluator cannot see the label — the explanation does not reveal its own incorrectness. An *unblinded* judge shown the ground truth produced a large spurious gap (e.g., completeness 3.51 vs. 2.22), which is an artifact of label leakage rather than a

[1] These estimates are robust: a newer judge (Opus-4.8), consensus-target calibration, a leakage-free snippet-level cross-validation, and stratum-balanced/contrastive-pair example selections all yield statistically equivalent agreement (within the ±0.1 sampling error at $n \approx 118$); full results are in the replication package.

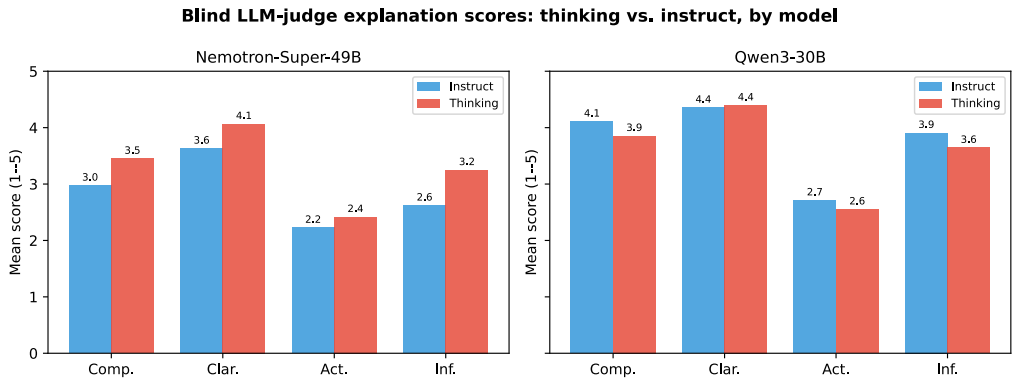


Figure 4: Calibrated blind LLM-judge (Sonnet-4.6) explanation scores (mean, 1–5; thinking vs. instruct) by model. The thinking-mode advantage is confined to Nemotron-Super-49B; for Qwen3-30B the two modes are statistically comparable, if anything marginally favouring instruct.

property of the explanations, and motivates the blind protocol. On this two-model frame, the thinking-mode benefit is also confined to Nemotron: Qwen3-30B shows no significant thinking–instruct difference on any dimension. After calibration, human–judge agreement reaches inter-human ceiling for four dimensions (adjacent agreement 77–100% across strata), strongest in correctly classified cases and weakest on false positives.

RQ3 Conclusion. Under a blinded protocol, both the automatic LLM judge and the human raters find thinking-mode explanations more *informative*; the judge additionally rates thinking higher on completeness and clarity, which humans do not corroborate—so that broader advantage is largely a property of the judge. It is further tempered on three fronts: substantially length-mediated (thinking's ∼4.6× longer explanations explain 40–60% of the gap), model-specific (Nemotron-Super-49B but not Qwen3-30B), and weakest on false positives, even though the calibrated judge matches the inter-human agreement ceiling and both evaluators rank the dimensions identically (clarity highest, actionability lowest). Evaluated blind, explanation quality also does not differ between correct and incorrect predictions—a fluent explanation does not signal its own correctness.

5 Discussion

5.1 Cost–Performance Trade-off

Table 8 summarizes the computational cost of thinking versus instruct mode across all four system configurations. Token counts use model-specific BPE tokenizers: Qwen for Qwen3 and Llama for Nemotron. Input tokens include system messages, task prompts, and accumulated conversation context across all agent turns; output tokens include all agent responses.

Thinking mode uses **2.7–4.4× more energy** and produces **2.5–3.6× more output tokens** than instruct mode. This compounds in MA, where thinking averages 25,533 tokens per sample (17,337 input + 8,196 output), versus 3,703 for NA. The costliest setting, MA with Nemotron-Super-49B thinking, consumes 36.4,kWh and costs $166.8 at RunPod H100 rates. Table 8 reports aggregate wall-clock GPU-time rather than per-sample end-to-end latency because runs were sharded and resumed across pods and sessions; per-sample latency would therefore mix model computation with scheduling and batching effects. Despite its higher cost, thinking mode does not consistently improve detection. In NA and SA, it raises F1 by only 0.10–0.11 while using 2.7–4.4× more energy; under DA, it slightly *lowers* mean F1 (−0.05; Table 5) despite nearly tripling energy use. MA shows mixed effects, with some gains and some degradations. Overall, the 64 configurations consumed about $2,448, 481 kWh of energy, and 423.0 million tokens.

Table 8: Cost–performance by system configuration and reasoning mode on PrimeVul Pair (870 samples). Duration, energy, and cost are totals over 8 configurations per cell (4 models × 2 prompts). "Dur." denotes aggregate wall-clock GPU-time, a throughput measure rather than per-sample latency because runs were sharded across pods and sessions. Tokens are reported per sample and in total; cost assumes RunPod H100 SXM at $2.69/GPU-hour.

		Total		Avg Tok./Sample		Total Tok.		Total
Design	Mode	Dur. (h)	kWh	In	Out	In (M)	Out (M)	Cost ($)
NA	Inst.	44.0	19.2	220	1,096	1.5	7.6	133
	Think	132.9	68.5	220	3,483	1.5	24.2	438
SA	Inst.	25.4	17.6	220	1,099	1.5	7.6	83
	Think	102.7	76.7	220	3,482	1.5	24.2	357
DA	Inst.	38.3	24.9	4,182	1,287	29.0	8.9	127
	Think	145.9	85.4	6,515	4,598	45.2	31.9	466
MA	Inst.	64.2	51.6	5,453	3,310	37.9	23.0	228
	Think	166.7	137.5	17,337	8,196	120.4	56.9	615
Total		**720**	**481**			**238.6**	**184.4**	**2,448**

Thinking mode offers the best cost–benefit trade-off in NA and SA, where extended reasoning influences the final prediction. In DA and MA, this benefit is diluted across agents while cost increases. Per configuration, SA averages 8.0 h, 2.2 M tokens, and $27.5; DA 11.5 h, 7.2 M tokens, and $37.1; and MA 14.4 h, 14.9 M tokens, and $52.7. Relative to SA, DA uses 3.3× more tokens and costs 1.35× more, while MA uses 6.8× more tokens and costs 1.9× more.

5.2 Collaboration versus Inference Budget, and Role Sensitivity

The RQ2 gains from orchestration could stem from collaboration itself or simply from a larger inference budget. To separate the two, we add call-matched single-agent baselines: Rev2 and Rev4 apply

Table 9: Budget-matched zero-shot comparison on 193 PrimeVul pairs by model and mode. Each cell reports F1 / P-C(%) from one run; "Calls" denotes per-sample inference budget. Four-call self-revision achieves the best or near-best P-C in every cell, while DA retains competitive F1 but weak P-C.

		Nemotron-49B		Qwen-30B	
Design	Calls	Inst. (F1/PC)	Think (F1/PC)	Inst. (F1/PC)	Think (F1/PC)
SA	1	.419 / 21.8	.624 / 22.9	.478 / 20.7	.516 / 22.3
DA	2	.657 / 1.0	.661 / 4.1	.666 / 0.5	.589 / 13.0
MA	4	.547 / 21.8	.489 / 23.8	.607 / 21.2	.528 / 20.7
Self-Rev (2)	2	.603 / 26.4	.596 / 18.1	.539 / 20.7	.512 / 23.3
Self-Rev (4)	4	.558 / 26.9	.604 / 21.8	.522 / 25.9	.518 / 24.3
Best-of-4	4	.484 / 21.8	.629 / 13.5	.541 / 16.6	.562 / 19.7
VulAgent-lite	6	.176 / 6.7	.615 / 9.3	.600 / 21.8	.594 / 17.1

deterministic self-revision with two and four calls (matching DA and MA), and Bon4 generates four responses at T=0.7 and takes the majority label (ties broken as vulnerable). To probe role sensitivity, VulAgent-lite adopts VulAgent's specialist hypothesis–validation workflow [52] without Joern-based CFG/DFG extraction, using four specialists, an aggregator, and a validator. All four configurations are evaluated on a pair-preserving 193-pair subset of PrimeVul Pair with the two largest models (Nemotron-Super-49B and Qwen-30B), both modes, and zero-shot (Table 9). Because each model–mode cell is a single run, we report these findings descriptively.

On these matched cells, four-call self-revision attains the highest mean P-C (24.7), modestly above SA and MA (both 21.9) and far above DA (4.7), at a much lower false-positive rate than DA (0.47 vs. 0.93), while DA's higher F1 stems from over-flagging rather than better discrimination. Two conclusions follow. First, DA's aggressive flagging destroys pairwise discrimination—its P-C collapses to 0.5–13.0 across cells despite competitive F1. Second, the four-agent MA's pairwise-discrimination edge does not exceed what a single agent achieves under the same call budget by critiquing and revising its own answer, indicating that much of the apparent collaboration benefit reflects additional inference budget rather than multi-agent structure itself. The alternative role design, VulAgent-lite [52], mainly shifts the false-positive rate rather than improving P-C and is unstable across families—it fails on Nemotron instruct (F1 = 0.176) while performing normally on Qwen—corroborating that orchestration primarily reshapes the alert burden rather than the underlying pairwise discrimination.

6 Threats to Validity

Internal Validity. Observed differences may reflect uncontrolled variation. We mitigated this by keeping the dataset, task, label space, response constraints, and prompt structure fixed, varying only the factor studied. Deterministic decoding ($T = 0$) reduced stochasticity but does not represent non-deterministic settings. Prompt sensitivity remains despite using identical structures and few-shot examples across models and modes. Agent design is another threat because many role, coordination, and aggregation choices exist; our configurations are representative, not task-optimal. To partially assess role sensitivity, we compared our debate-oriented MA with VulAgent-lite (Table 9), which mainly changed alert burden rather than pairwise discrimination.

Construct Validity. Our metrics may not fully capture the intended constructs. Accuracy, precision, recall, F1, and P-C cover instance-level classification and pairwise discrimination; stability measures cross-setting variability and improvement consistency across models, prompts, and reasoning modes; and explanation quality is assessed through completeness, informativeness, clarity, and actionability. We reduce subjectivity through two blinded annotators, detailed guidelines, calibration examples, and complementary LLM-based assessment, although interpretation remains partly subjective. The rubric measures developer-facing explanation quality, not faithfulness or causal validity: fluent, well-structured explanations may not reflect the model's actual reasoning, and blinded evaluators cannot distinguish correct from incorrect explanations (Section 4.3). The single external Claude judge may have stylistic preferences and residual sensitivity to response length. We mitigate these threats through blinding, human calibration, validation against the inter-human agreement ceiling, robustness checks with an alternative judge and exemplar settings, and by treating automatic scores as complementary to human ratings, especially where agreement is weakest (clarity and false positives).

External Validity. Generalizability is limited by the task, dataset, and models. We study only vulnerability detection on PrimeVul Pair, so results may not transfer to other SE tasks such as repair or code generation. The benchmark supports controlled function-level discrimination between lexically similar vulnerable and benign code but omits project context, interprocedural reasoning, and repository-level vulnerabilities. Our four open Nemotron and Qwen models provide comparable instruct/thinking modes within our hardware budget, but do not represent the wider LLM landscape; larger models such as DeepSeek-V3.1 and Kimi-K2 require greater multi-GPU resources. Future work should evaluate additional reasoning-oriented families, including distilled DeepSeek-R1 and Gemma variants, and extend to costlier repository-level settings requiring dependency resolution, retrieval, and tool support.

Conclusion Validity. The single-run, temperature-0 setup supports controlled comparison by avoiding sampling variance, but limits conclusions to deterministic decoding and does not capture repeated-run variability. For RQ3, manual evaluation uses a blinded sample covering correct and incorrect predictions, introducing sampling uncertainty; we partly mitigate this by applying a calibrated automatic judge, validated against human ratings at the inter-annotator agreement ceiling, to the full evaluation frame.

7 Conclusion

This paper presented a controlled study of reasoning modes, agentic orchestration, and explanation quality in LLM-based vulnerability detection. On PrimeVul Pair, thinking improves non-agent F1 in 7 of 8 settings, mainly through recall, but discrimination remains limited. DA achieves the highest F1/recall while flagging 88.2% of functions, making it a high-recall screen; MA offers better pairwise discrimination at a lower false-positive rate. Budget-matched self-revision can match or exceed MA on P-C, showing that some agentic gains stem from additional inference budget. Explanation gains are model-specific, largely length-mediated, and do not indicate prediction correctness. Thinking uses 2.7–4.4× more energy and 2.5–3.6× more output tokens. Overall, reasoning and orchestration involve objective- and budget-dependent trade-offs under deterministic, function-level evaluation.

Acknowledgments

The work has been conducted as part of the ENFIELD project (101120657), the INTEND project (101135576), and the SECASSURED project (101225858) funded by the European Commission within the HEU Programme, and the Privacy@Edge project (338909) funded by the Research Council of Norway's IKTPLUS-ICT and digital innovation Programme.

Data Availability Statement

The replication package is publicly available on Zenodo at https://doi.org/10.5281/zenodo.21401826.

References

[1] AG2 (formerly AutoGen). 2026. AG2: Open-Source AgentOS for AI Agents. https://github.com/ag2ai/ag2

[2] Md Basim Uddin Ahmed, Nima Shiri Harzevili, Jiho Shin, Hung Viet Pham, and Song Wang. 2025. SecVulEval: Benchmarking LLMs for real-world C/C++ vulnerability detection. *arXiv preprint arXiv:2505.19828* (2025).

[3] Md Tauseef Alam, Raju Halder, and Abyayananda Maiti. 2024. Detection made easy: Potentials of large language models for solidity vulnerabilities. *arXiv preprint arXiv:2409.10574* (2024).

[4] Merve Astekin, Arda Goknil, Sagar Sen, Simeon Tverdal, and Phu Nguyen. 2025. Detecting Technical Debt in Source Code Changes Using Large Language Models. In *International Conference on Product-Focused Software Process Improvement*. Springer, 334–352.

[5] Md Abdul Awal, Mrigank Rochan, and Chanchal K Roy. 2025. Investigating adversarial attacks in software analytics via machine learning explainability. *Software Quality Journal* 33, 3 (2025), 31.

[6] Biagio Boi, Christian Esposito, and Sokjoon Lee. 2024. Smart contract vulnerability detection: The role of large language model (LLM). *ACM SIGAPP applied computing review* 24, 2 (2024), 19–29.

[7] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2025. Repairagent: An autonomous, LLM-based agent for program repair. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2188–2200.

[8] Di Cao, Yong Liao, and Xiuwei Shang. 2024. Realvul: Can we detect vulnerabilities in web applications with LLM?. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 8268–8282.

[9] Sicong Cao, Xiaobing Sun, Ratnadira Widyasari, David Lo, Xiaoxue Wu, Lili Bo, Jiale Zhang, Bin Li, Wei Liu, Di Wu, et al. 2025. A Systematic Literature Review on Explainability for ML/DL-based Software Engineering. *Comput. Surveys* (2025).

[10] Zichen Chen, Jianda Chen, Ambuj Singh, and Misha Sra. 2024. XplainLLM: A knowledge-augmented dataset for reliable grounded explanations in LLMs. In *Proceedings of the 2024 conference on empirical methods in natural language processing*. 7578–7596.

[11] Hoa Khanh Dam, Truyen Tran, and Aditya Ghose. 2018. Explainable software analytics. In *Proceedings of the 40th international conference on software engineering: New ideas and emerging results*. 53–56.

[12] Helmut Degen, Christof Budnik, Gregory Conte, Andrew Lintereur, and Seth Weber. 2022. How to explain it to energy engineers? A qualitative user study about trustworthiness, understandability, and actionability. In *International Conference on Human-Computer Interaction*. Springer, 262–284.

[13] Jay DeYoung, Sarthak Jain, Nazneen Fatema Rajani, Eric Lehman, Caiming Xiong, Richard Socher, and Byron C Wallace. 2020. ERASER: A benchmark to evaluate rationalized NLP models. In *Proceedings of the 58th annual meeting of the association for computational linguistics*. 4443–4458.

[14] Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, and Yizheng Chen. 2024. Vulnerability detection with code language models: How far are we? *arXiv preprint arXiv:2403.18624* (2024).

[15] Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, and Yizheng Chen. 2025. Vulnerability Detection with Code Language Models: How Far are We?. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. 1729–1741. doi:10.1109/ICSE55347.2025.00038

[16] Xueying Du, Geng Zheng, Kaixin Wang, Yi Zou, Yujia Wang, Wentai Deng, Jiayi Feng, Mingwei Liu, Bihuan Chen, Xin Peng, et al. 2024. Vul-rag: Enhancing LLM-based vulnerability detection via knowledge-level rag. *ACM Transactions on Software Engineering and Methodology* (2024).

[17] Sarah Elder, Md Rayhanur Rahman, Gage Fringer, Kunal Kapoor, and Laurie Williams. 2024. A survey on software vulnerability exploitability assessment. *Comput. Surveys* 56, 8 (2024), 1–41.

[18] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M Zhang. 2023. Large language models for software engineering: Survey and open problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. IEEE, 31–53.

[19] Chongzhou Fang, Ning Miao, Shaurya Srivastav, Jialin Liu, Ruoyu Zhang, Ruijie Fang, Ryan Tsang, Najmeh Nazari, Han Wang, Houman Homayoun, et al. 2024. Large language models for code analysis: Do LLMs really do their job?. In *33rd USENIX Security Symposium (USENIX Security 24)*. 829–846.

[20] Mohamed Amine Ferrag, Ammar Battah, Norbert Tihanyi, Ridhi Jain, Diana Maimuţ, Fatima Alwahedi, Thierry Lestable, Narinderjit Singh Thandi, Abdechakour Mechri, Merouane Debbah, et al. 2025. SecureFalcon: Are we there yet in automated software vulnerability detection with LLMs? *IEEE Transactions on Software Engineering* (2025).

[21] Michael Fu, Jirat Pasuksmit, and Chakkrit Tantithamthavorn. 2025. Ai for devsecops: A landscape and future opportunities. *ACM Transactions on Software Engineering and Methodology* 34, 4 (2025), 1–61.

[22] Michael Fu, Chakkrit Kla Tantithamthavorn, Van Nguyen, and Trung Le. 2023. Chatgpt for vulnerability detection, classification, and repair: How far are we?. In *2023 30th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 632–636.

[23] Diana Galvan-Sosa, Gabrielle Gaudeau, Pride Kavumba, Yunmeng Li, Hongyi Gu, Zheng Yuan, Keisuke Sakaguchi, and Paula Buttery. 2025. Rubrik's cube: Testing a new rubric for evaluating explanations on the CUBE dataset. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 23800–23839.

[24] Cuiyun Gao, Xing Hu, Shan Gao, Xin Xia, and Zhi Jin. 2025. The current challenges of software engineering in the era of large language models. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–30.

[25] Zeyu Gao, Hao Wang, Yuchen Zhou, Wenyu Zhu, and Chao Zhang. 2023. How far have we gone in vulnerability detection using large language models. *arXiv preprint arXiv:2311.12420* (2023).

[26] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, et al. 2024. A survey on LLM-as-a-judge. *The Innovation* (2024).

[27] Junda He, Jieke Shi, Terry Yue Zhuo, Christoph Treude, Jiamou Sun, Zhenchang Xing, Xiaoning Du, and David Lo. 2026. LLM-as-a-Judge for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Transactions on Software Engineering and Methodology* (2026). doi:10.1145/3797276

[28] Junda He, Christoph Treude, and David Lo. 2025. LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–30.

[29] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–79.

[30] Xing Hu, Feifei Niu, Junkai Chen, Xin Zhou, Junwei Zhang, Junda He, Xin Xia, and David Lo. 2025. Assessing and advancing benchmarks for evaluating large language models in software engineering tasks. *ACM Transactions on Software Engineering and Methodology* (2025).

[31] Alon Jacovi and Yoav Goldberg. 2020. Towards faithfully interpretable NLP systems: How should we define and evaluate faithfulness?. In *Proceedings of the 58th annual meeting of the association for computational linguistics*. 4198–4205.

[32] Haolin Jin, Linghan Huang, Haipeng Cai, Jun Yan, Bo Li, and Huaming Chen. 2024. From LLMs to LLM-based agents for software engineering: A survey of current, challenges and future. *arXiv preprint arXiv:2408.02479* (2024).

[33] Avishree Khare, Saikat Dutta, Ziyang Li, Alaia Solko-Breslin, Rajeev Alur, and Mayur Naik. 2025. Understanding the effectiveness of large language models in detecting security vulnerabilities. In *2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 103–114.

[34] Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. 2024. Large language model-based agents for software engineering: A survey. *ACM Transactions on Software Engineering and Methodology* (2024).

[35] Yue Liu, Sin Kit Lo, Qinghua Lu, Liming Zhu, Dehai Zhao, Xiwei Xu, Stefan Harrer, and Jon Whittle. 2025. Agent design pattern catalogue: A collection of architectural patterns for foundation model based agents. *Journal of Systems and Software* 220 (2025), 112278.

[36] Meike Nauta, Jan Trienes, Shreyasi Pathak, Elisa Nguyen, Michelle Peters, Yasmin Schmitt, Jörg Schlötterer, Maurice Van Keulen, and Christin Seifert. 2023. From anecdotal evidence to quantitative evaluation methods: A systematic review on evaluating explainable ai. *Comput. Surveys* 55, 13s (2023), 1–42.

[37] Adela Nedisan Videsjorden, Hui Song, Arda Goknil, Dumitru Roman, and Ahmet Soylu. 2026. LUMEN: Enhancing IoT system observability with multi-agent large language models and knowledge graphs. *ACM Transactions on Internet of Things* 7, 2 (2026), 1–40.

[38] Khouloud Oueslati, Gabriel Laberge, Maxime Lamothe, and Foutse Khomh. 2024. Mining action rules for defect reduction planning. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 2309–2331.

[39] Niklas Risse, Jing Liu, and Marcel Böhme. 2025. Top score on the wrong exam: On benchmarking in machine learning for vulnerability detection. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 388–410.

[40] Ze Sheng, Zhicheng Chen, Shuning Gu, Heqing Huang, Guofei Gu, and Jeff Huang. 2025. LLMs in software security: A survey of vulnerability detection techniques and insights. *Comput. Surveys* 58, 5 (2025), 1–35.
[41] Aleksei Shestov, Rodion Levichev, Ravil Mussabayev, Evgeny Maslov, Pavel Zadorozhny, Anton Cheshkov, Rustam Mussabayev, Alymzhan Toleu, Gulmira Tolegen, and Alexander Krassovitskiy. 2025. Finetuning large language models for vulnerability detection. *IEEE Access* (2025).
[42] Benjamin Steenhoek, Md Mahbubur Rahman, Monoshi Kumar Roy, Mirza Sanjida Alam, Hengbo Tong, Swarna Das, Earl T Barr, and Wei Le. 2024. To err is machine: Vulnerability detection challenges LLM reasoning. *arXiv preprint arXiv:2403.17218* (2024).
[43] Chi Sun, Yong Li, and Zheyuan Pan. 2025. SDPFE: A Formal Reasoning Approach for Explaining Software Defect Predictions. In *2025 25th International Conference on Software Quality, Reliability, and Security Companion (QRS-C)*. IEEE, 168–177.
[44] Karl Tamberg and Hayretdin Bahsi. 2025. Harnessing large language models for software vulnerability detection: A comprehensive benchmarking study. *IEEE Access* (2025).
[45] Hua Beng Tan, Ratnadira Widyasari, Merve Astekin, Arda Goknil, Hasan Sözer, Yan Naing Tun, Erik Johannes Husom, Lwin Khin Shar, and Lingxiao Jiang. 2026. Replication Package for "To Think or Not to Think: Evaluating LLM Reasoning and Agents in Vulnerability Detection". doi:10.5281/zenodo.21401826
[46] Chakkrit Kla Tantithamthavorn and Jirayus Jiarpakdee. 2021. Explainable AI for software engineering. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1–2.
[47] K.-T. Tran, D. Dao, M.-D. Nguyen, Q.-V. Pham, B. O'Sullivan, and H.D. Nguyen. 2025. Multi-Agent Collaboration Mechanisms: A Survey of LLMs. arXiv:2501.06322 [cs.AI] https://arxiv.org/abs/2501.06322
[48] Saad Ullah, Mingji Han, Saurabh Pujar, Hammond Pearce, Ayse Coskun, and Gianluca Stringhini. 2024. LLMs cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks. In *2024 IEEE symposium on security and privacy (SP)*. IEEE, 862–880.
[49] Mythreyi Velmurugan, Chun Ouyang, Yue Xu, Renuka Sindhgatta, Bemali Wickramanayake, and Catarina Moreira. 2025. Developing guidelines for functionally-grounded evaluation of explainable artificial intelligence using tabular data. *Engineering Applications of Artificial Intelligence* 141 (2025), 109772.
[50] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering* 50, 4 (2024), 911–936.
[51] Yanlin Wang, Wanjun Zhong, Yanxian Huang, Ensheng Shi, Min Yang, Jiachi Chen, Hui Li, Yuchi Ma, Qianxiang Wang, and Zibin Zheng. 2025. Agents in software engineering: Survey, landscape, and vision. *Automated Software Engineering* 32, 2 (2025), 70.
[52] Z. Wang, G. Li, J. Li, H. Zhu, and Z. Jin. 2026. VulAgent: Hypothesis-Validation Driven Multi-Agent Architecture for Vulnerability Detection. In *Findings of the Association for Computational Linguistics: ACL 2026*, M. Liakata, V. P. Moreira, J. Zhang, and D. Jurgens (Eds.). Association for Computational Linguistics, 18598–18616. doi:10.18653/v1/2026.findings-acl.928
[53] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
[54] Martin Weyssow, Chengran Yang, Junkai Chen, Ratnadira Widyasari, Ting Zhang, Huihui Huang, Huu Hung Nguyen, Yan Naing Tun, Tan Bui, Yikun Li, et al. 2025. R2vul: Learning to reason about software vulnerabilities with reinforcement learning and structured reasoning distillation. *arXiv preprint arXiv:2504.04699* (2025).
[55] Ratnadira Widyasari, Martin Weyssow, Ivana Clairine Irsan, Han Wei Ang, Frank Liauw, Eng Lieh Ouh, Lwin Khin Shar, Hong Jin Kang, and David Lo. 2025. Let the trial begin: A mock-court approach to vulnerability detection using llm-based agents. *arXiv preprint arXiv:2505.10961* (2025).
[56] Ratnadira Widyasari, Ting Zhang, Abir Bouraffa, Walid Maalej, and David Lo. 2025. Explaining explanations: An empirical study of explanations in code reviews. *ACM Transactions on Software Engineering and Methodology* 34, 6 (2025), 1–30.
[57] Robert F Woolson. 2007. Wilcoxon signed-rank test. *Wiley encyclopedia of clinical trials* (2007), 1–3.
[58] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 801–824.
[59] Guifang Xu, Chengbin Feng, Xingcheng Guo, Zhiling Zhu, and Wei Wang. 2025. A joint learning framework for bridging defect prediction and interpretation. *IEEE Transactions on Reliability* (2025).
[60] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems* 37 (2024), 50528–50652.
[61] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*.
[62] Alperen Yildiz, Sin G Teo, Yiling Lou, Yebo Feng, Chong Wang, and Dinil Mon Divakaran. 2025. Benchmarking LLMs and LLM-based agents in practical vulnerability detection for code repositories. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 30848–30865.
[63] Jinqiang Yu, Michael Fu, Alexey Ignatiev, Chakkrit Tantithamthavorn, and Peter Stuckey. 2024. A formal explainer for just-in-time defect predictions. *ACM Transactions on Software Engineering and Methodology* 33, 7 (2024), 1–31.
[64] Jerrold H Zar. 2005. Spearman rank correlation. *Encyclopedia of biostatistics* 7 (2005).
[65] Haiyan Zhao, Hanjie Chen, Fan Yang, Ninghao Liu, Huiqi Deng, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, and Mengnan Du. 2024. Explainability for large language models: A survey. *ACM Transactions on Intelligent Systems and Technology* 15, 2 (2024), 1–38.
[66] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging LLM-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems* 36 (2023), 46595–46623.
[67] Jianlong Zhou, Amir H Gandomi, Fang Chen, and Andreas Holzinger. 2021. Evaluating the quality of machine learning explanations: A survey on methods and metrics. *Electronics* 10, 5 (2021), 593.
[68] Xin Zhou, Sicong Cao, Xiaobing Sun, and David Lo. 2025. Large language model for vulnerability detection and repair: Literature review and the road ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–31.
[69] Xin Zhou, Ting Zhang, and David Lo. 2024. Large language model for vulnerability detection: Emerging results and future directions. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*. 47–51.

Received 2026-03-26; accepted 2026-06-18